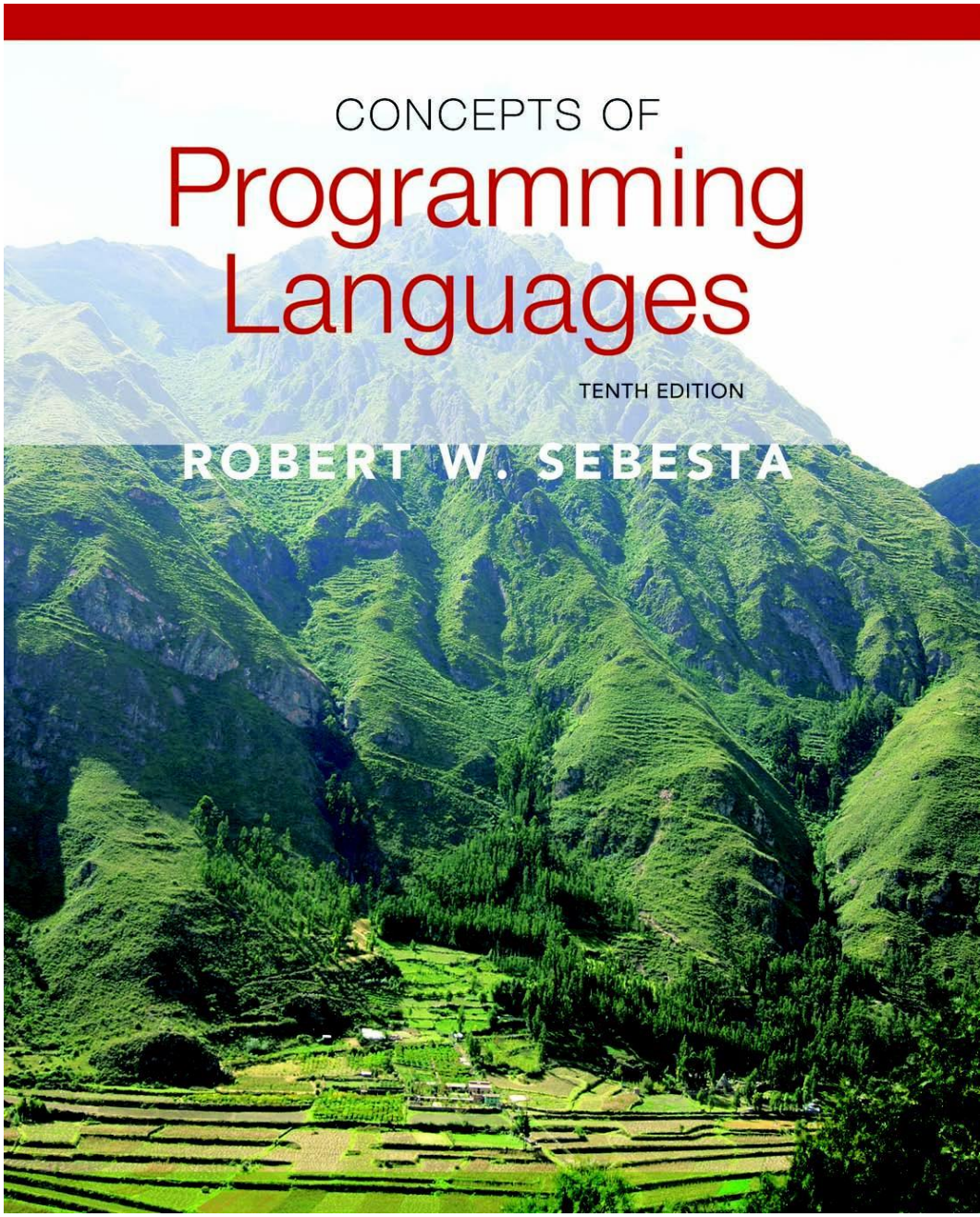
Chapter 7

Expressions and Assignment Statements



Chapter 7 Topics

- Introduction
- Arithmetic Expressions
- Overloaded Operators
- Type Conversions
- Relational and Boolean Expressions
- Short-Circuit Evaluation
- Assignment Statements
- Mixed-Mode Assignment

Copyright © 2012 Addison-Wesley. All rights reserved.

Introduction

- Expressions are the fundamental means of specifying computations in a programming language
- To understand expression evaluation, need to be familiar with the orders of operator and operand evaluation
- Essence of imperative languages is dominant role of assignment statements

Copyright © 2012 Addison-Wesley. All rights reserved.

Arithmetic Expressions

- Arithmetic evaluation was one of the motivations for the development of the first programming languages
- Arithmetic expressions consist of operators, operands, parentheses, and function calls

Copyright © 2012 Addison-Wesley. All rights reserved.

Arithmetic Expressions: Design Issues

- Design issues for arithmetic expressions
  - Operator precedence rules?
  - Operator associativity rules?
  - Order of operand evaluation?
  - Operand evaluation side effects?
  - Operator overloading?
  - Type mixing in expressions?

Copyright © 2012 Addison-Wesley. All rights reserved.

Arithmetic Expressions: Operators

- A unary operator has one operand
- A binary operator has two operands
- A ternary operator has three operands

Copyright © 2012 Addison-Wesley. All rights reserved.

Arithmetic Expressions: Operator Precedence Rules

- The *operator precedence rules* for expression evaluation define the order in which "adjacent" operators of different precedence levels are evaluated
- Typical precedence levels
  - parentheses
  - unary operators
  - ** (*exponentiation*, if the language supports)
  - *, /
  - +, -

Copyright © 2012 Addison-Wesley. All rights reserved.

Arithmetic Operator Precedence Rules in Ruby and C-based Languages

	Ruby	*C-Based Languages*
Highest	**	postfix ++, --
	unary +, -	prefix ++, --, unary +, -
	*, /, %	*, /, %
Lowest	binary +, -	binary +, -

Copyright © 2012 Addison-Wesley. All rights reserved.

Arithmetic Expressions: Operator Associativity Rule

- The *operator associativity rules* for expression evaluation define the order in which adjacent operators with the same precedence level are evaluated
- Typical associativity rules
  - Left to right, except **, which is right to left

- APL is different; all operators have equal precedence and all operators associate right to left
- Precedence and associativity rules can be overridden with parentheses

Copyright © 2012 Addison-Wesley. All rights reserved.

Operator Associativity Rules in Ruby and C-based Languages

Language	*Associativity Rule*
Ruby	Left: *, /, +, -
	Right: **
C-based languages	Left: *, /, %, binary +, binary -
	Right: ++, --, unary -, unary +

Copyright © 2012 Addison-Wesley. All rights reserved.

Expressions in Ruby and LISP

- Ruby
  - All arithmetic, relational, and assignment operators, as well as array indexing, shifts, and bit-wise logic operators, are implemented as methods
  - One result of this is that these operators can all be overriden by application programs
- LISP
  - All arithmetic and logic operations are by explicitly called subprograms
  - `a + b * c` is coded as `(+ a (* b c))`

Copyright © 2012 Addison-Wesley. All rights reserved.

Arithmetic Expressions: Conditional Expressions

- Conditional Expressions
  - C-based languages (e.g., C, C++)
  - An example:

```
average = (count == 0)? 0 : sum / count
```

  - Evaluates as if written as follows:

```
if (count == 0)
   average = 0
else
   average = sum /count
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Arithmetic Expressions: Operand Evaluation Order

- *Operands*
  1. Variables: fetch the value from memory
  2. Constants: sometimes a fetch from memory; sometimes the constant is in the machine language instruction
  3. Parenthesized expressions: evaluate all operands and operators first
  4. The most interesting case is when an operand is a function call

Copyright © 2012 Addison-Wesley. All rights reserved.

✅ C Operator Precedence and Associativity Table

Precedence	Operator(s)	Description	Associativity	Operand Evaluation Order
1 (Highest)	`()` `[]` `->` `.`	Function call, array index, structure access	Left-to-right	**Unspecified** (e.g. function arguments)
	`++` `--` (postfix)	Post-increment/decrement	Left-to-right	Operand before increment evaluated first
2	`++` `--` `+` `-` `!` `~` `*` `&` `(type)` `sizeof` `_Alignof`	Unary operators	Right-to-left	Single operand, so N/A
3	`*` `/` `%`	Multiplication, division, modulus	Left-to-right	**Unspecified**
4	`+` `-`	Addition, subtraction	Left-to-right	**Unspecified**
5	`<<` `>>`	Bit shifts	Left-to-right	**Unspecified**
6	`<` `<=` `>` `>=`	Relational operators	Left-to-right	**Unspecified**
7	`==` `!=`	Equality operators	Left-to-right	**Unspecified**
8	`&`	Bitwise AND	Left-to-right	**Unspecified**
9	`^`	Bitwise XOR	Left-to-right	**Unspecified**
10	`|`	Bitwise OR	Left-to-right	**Unspecified**
11	`&&`	Logical AND	Left-to-right	**Defined** (left-to-right, short-circuits)
12	`||`	Logical OR	Left-to-right	**Defined** (left-to-right, short-circuits)
13	`?:`	Ternary conditional	**Right-to-left**	Condition → then/else (only one evaluated)
14	`=` `+=` `-=` `*=` etc.	Assignment operators	**Right-to-left**	**Unspecified** evaluation of RHS

Arithmetic Expressions: Potentials for Side Effects

- *Functional side effects:* when a function changes a two-way parameter or a non-local variable
- Problem with functional side effects:
  - When a function referenced in an expression alters another operand of the expression; e.g., for a parameter change:

```
a = 10;
/* assume that fun changes its parameter */
b = a + fun(&a);
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Functional Side Effects

- Effect of order of evaluation of the operands in the expression → `a + fun1()`
  - if a is evaluated first → `a=8`
  - if the function call is evaluated first → `a=20`

```
int a = 5;
int fun1() {
        a = 17;
        return 3;
} /* end of fun1 */
void main() {
        a = a + fun1();
} /* end of main */
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Functional Side Effects

- Two possible solutions to the problem
  1. Write the language definition to disallow functional side effects
     - No two-way parameters in functions
     - No non-local references in functions
     - **Advantage**: it works!
     - **Disadvantage**: inflexibility of one-way parameters and lack of non-local references
  2. Write the language definition to demand that operand evaluation order be fixed
     - **Disadvantage**: limits some compiler optimizations
     - Java requires that operands appear to be evaluated in left-to-right order

Copyright © 2012 Addison-Wesley. All rights reserved.

Referential Transparency

- A program has the property of *referential transparency* if any two expressions in the program that have the same value can be substituted for one another anywhere in the program, without affecting the action of the program

```
result1 = (fun(a) + b) / (fun(a) - c);
temp = fun(a);
result2 = (temp + b) / (temp - c);
```

If `fun` has no side effects, `result1 = result2`

Otherwise, not, and referential transparency is violated

Copyright © 2012 Addison-Wesley. All rights reserved.

Referential Transparency (continued)

- Advantage of referential transparency
  - Semantics of a program is much easier to understand if it has referential transparency
- Because they do not have variables, programs in pure functional languages are referentially transparent
  - Functions cannot have state, which would be stored in local variables
  - If a function uses an outside value, it must be a constant (there are no variables). So, the value of a function depends only on its parameters

Copyright © 2012 Addison-Wesley. All rights reserved.

Overloaded Operators

- Use of an operator for more than one purpose is called *operator overloading*
- Some are common (e.g., `+` for `int` and `float`)
- Some are potential trouble (e.g., `*` in C and C++)
  - Loss of compiler error detection (omission of an operand should be a detectable error)
  - Some loss of readability

Copyright © 2012 Addison-Wesley. All rights reserved.

Type Conversions

- A *narrowing conversion* is one that converts an object to a type that cannot include all the values of the original type

  e.g., `float` to `int`

- A *widening conversion* is one in which an object is converted to a type that can include at least approximations to all the values of the original type

  e.g., `int` to `float`

Copyright © 2012 Addison-Wesley. All rights reserved.

Type Conversions: Mixed Mode

- A *mixed-mode expression* is one that has operands of different types

- A ***coercion*** is an implicit type conversion
- Disadvantage of coercions:
  - They decrease the type error detection ability of the compiler

- In most languages, all numeric types are coerced in expressions, using widening conversions
- In ML and F#, there are no coercions in expressions

Copyright © 2012 Addison-Wesley. All rights reserved.

Type Conversions: Mixed Mode

```
int a;
float b, c, d;
. . .
d = b * a;
```

Compiler will coerce
the value of the `int` operand, `a`, to `float`.

Copyright © 2012 Addison-Wesley. All rights reserved.

Explicit Type Conversions

- Called *casting* in C-based languages
- Examples
  - `C: (int)angle`
  - `F#: float(sum)`

Note that F#'s syntax is similar to that of function calls

Copyright © 2012 Addison-Wesley. All rights reserved.

Errors in Expressions

- Causes
  - Inherent limitations of arithmetic
    e.g., division by zero
  - Limitations of computer arithmetic
    e.g. overflow
- Often ignored by the run-time system

Copyright © 2012 Addison-Wesley. All rights reserved.

Relational and Boolean Expressions

- Relational Expressions
  - Use relational operators and operands of various types
  - Evaluate to some Boolean representation
  - Operator symbols used vary somewhat among languages (`!=`, `/=`, `~=`, `.NE.`, `<>`, `#`)

- JavaScript and PHP have two additional relational operator, `===` and `!==`
  - Similar to their cousins, `==` and `!=`, except that they do not coerce their operands
  - Ruby uses `==` for equality relation operator that uses coercions and `eql?` for those that do not

Copyright © 2012 Addison-Wesley. All rights reserved.

Relational and Boolean Expressions

- Boolean Expressions
  - Operands are Boolean and the result is Boolean
- C99 has no Boolean type--it uses **`int`** type with 0 for false and 1 for true
- One odd characteristic of C's expressions:
  - **`a < b < c`** is a legal expression, but the result is not what you might expect:
  - Left operator is evaluated, producing 0 or 1
  - The evaluation result is then compared with the third operand (i.e., **`c`**)

Copyright © 2012 Addison-Wesley. All rights reserved.

Short Circuit Evaluation

- An expression in which the result is determined without evaluating all the operands and/or operators

- Example: `(13 * a) * (b / 13 - 1)`
  If `a` is zero, there is no need to evaluate `(b /13 - 1)`

- Problem with non-short-circuit evaluation

```
index = 0;
while (index < length) && (LIST[index] != value)
      index++;
```

  - When `index=length, LIST[index]` will cause an indexing problem (assuming `LIST` is `length - 1` long)

Copyright © 2012 Addison-Wesley. All rights reserved.

Short Circuit Evaluation (continued)

- **C, C++, and Java**: use short-circuit evaluation for the usual Boolean operators (`&&` and `||`), but also provide bitwise Boolean operators that are not short circuit (`&` and `|`)

- All logic operators in **Ruby, Perl, ML, F#, and Python** are short-circuit evaluated

- Short-circuit evaluation exposes the potential problem of side effects in expressions
  e.g. `(a > b) || (b++ / 3)`

Copyright © 2012 Addison-Wesley. All rights reserved.

Assignment Statements

- The general syntax

  `<target_var> <assign_operator> <expression>`

- The assignment operator

  = Fortran, BASIC, the C-based languages

  := ALGOL 60

Copyright © 2012 Addison-Wesley. All rights reserved.

Assignment Statements: Conditional Targets

- Conditional targets (Perl)

```
($flag ? $total : $subtotal) = 0
```

Which is equivalent to

```
if ($flag){
  $total = 0
} else {
  $subtotal = 0
}
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Assignment Statements: Compound Assignment Operators

- A shorthand method of specifying a commonly needed form of assignment
- Introduced in ALGOL; adopted by C and the C-based languages
  - Example

```
a = a + b
```

can be written as

```
a += b
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Assignment Statements: Unary Assignment Operators

- Unary assignment operators in C-based languages combine increment and decrement operations with assignment

- Examples

  `sum = ++count` (`count` incremented, then assigned to `sum`)

  `sum = count++` (`count` assigned to `sum`, then incremented

  `count++` (`count` incremented)

Copyright © 2012 Addison-Wesley. All rights reserved.

Assignment as an Expression

- In the C-based languages, Perl, and JavaScript, the assignment statement produces a result and can be used as an operand

```
while ((ch = getchar())!= EOF){...}
```

  - `ch = getchar()` is carried out;
  - the result (assigned to `ch`) is used as a conditional value for the `while` statement

- Disadvantage: another kind of expression side effect

Copyright © 2012 Addison-Wesley. All rights reserved.

Multiple Assignments

- Perl, Ruby, and Lua allow multiple-target multiple-source assignments

```
($first, $second, $third) = (20, 30, 40);
```

Also, the following is legal and performs an interchange:

```
($first, $second) = ($second, $first);
```

- Python:

```
c = (1, 2)
a, b = c
```

Copyright © 2012 Addison-Wesley. All rights reserved.